



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Administración y Organización de Datos

No. Alumno:

Christian Daniel Martínez Márquez

No. Control:

242310396

Docente:

Jesús Salas Marín

1. El Enfoque del Proyecto: Del Dato Bruto a la Toma de Decisiones

En este tercer corte, el reto subió bastante de nivel. Ya no se trataba solo de validar usuarios o mover archivos de un lado a otro, sino de enfrentarse a un problema crítico en cualquier empresa: el análisis de datos masivos. Para este proyecto, el objetivo principal fue construir un sistema capaz de tomar un archivo con información cruda de ventas de tecnología, procesarlo en segundos de forma automática, generar métricas clave y presentárselo a los directivos en una interfaz web limpia y visual. La idea clave aquí fue la automatización completa, logrando que el sistema haga todo el trabajo pesado sin que el usuario tenga que mover un solo dedo.

2. Procesamiento de Datos y Ciencia de Datos con Python

El verdadero cerebro analítico del proyecto está en el backend y corre a cargo de dos scripts de Python: Reportes.py y Graficas.py. Todo el flujo de datos arranca con ventas_tecnologia.csv, que es el archivo donde se encuentra guardado el historial de ventas de la empresa.

El script Reportes.py se encarga de abrir este archivo .csv, leer la información estructurada y realizar los cálculos matemáticos para extraer totales, promedios y rendimientos. El resultado de este procesamiento se escribe de forma automática en un archivo limpio llamado reporte_resultado.txt. Por otro lado, para que los números no se queden solo en texto, entra en acción Graficas.py. Usando librerías especializadas en visualización de datos, este script genera de manera automatizada tres gráficos de nivel profesional: ventas_mes.png (para ver el comportamiento del tiempo), ventas_producto.png y porcentaje_producto.png (para identificar qué tecnologías están generando más ganancias). Con esto, demostramos el control de flujos I/O y manipulación estadística avanzada.

3. Integración de la Capa Web y Visualización Dinámica en PHP

Para llevar estos reportes directamente al navegador y que cualquier administrativo pueda verlos, desarrollé la capa de presentación utilizando tres archivos clave: verReporte.py, verReporte.php y verReporteSeparado.php.

La integración funciona de una forma genial: el archivo verReporte.php actúa como el portal principal que unifica la solución. Desde ahí, el servidor PHP manda a llamar los análisis en segundo plano (usando los scripts de Python). Una vez que el backend genera los datos y las imágenes, PHP se encarga de renderizar todo dinámicamente. Para dar flexibilidad, creé también verReporteSeparado.php, que permite segmentar la información para que los usuarios puedan consultar módulos de gráficos o datos financieros de forma independiente según lo que

necesiten en el momento. Esto demuestra una arquitectura limpia donde la interfaz web no está sobrecargada, sino que actúa como un visor inteligente de los procesos de fondo.

4. Estructura Estética y Experiencia de Usuario (CSS)

Aprovechando que en este corte manejamos contenido muy visual (como las gráficas generadas de las ventas), el diseño se estructuró para ser sumamente limpio y corporativo, con una distribución fluida que resalta las imágenes sobre fondos neutros y elegantes. El diseño de las tablas donde se muestran las métricas financieras tiene un estilo moderno, con tipografías muy legibles y un espaciado equilibrado para evitar la fatiga visual. Los botones para alternar entre reportes o activar la actualización de las gráficas fueron estilizados con efectos visuales sutiles al pasar el cursor, logrando que el sistema se sienta responsivo, vivo y listo para una presentación ejecutiva.

5. Conclusión y Escalabilidad en el Entorno Real

Haber desarrollado este sistema me dio una perspectiva tremenda de lo que se hace en el mundo real. Aprendí a conectar el potencial analítico y científico de Python con la agilidad de despliegue que ofrece PHP en la web. Es un proyecto que demuestra cómo optimizar recursos, conectar herramientas multiplataforma y transformar datos planos en herramientas que le sirven a los jefes para tomar decisiones estratégicas.

El proyecto está diseñado de forma modular, lo que significa que es totalmente escalable. Si la empresa decidiera implementar este sistema en producción para analizar millones de ventas globales en tiempo real, el cambio sería muy natural: la lógica que actualmente lee el archivo .csv se reconfiguraría para conectarse directamente a un Data Warehouse o a una base de datos analítica como BigQuery o PostgreSQL, y los scripts de Python se convertirían en microservicios independientes. Toda la interfaz y la lógica de renderizado web en PHP que ya construí se mantendrían funcionando perfectamente.